



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



## **REPORTE DE PRÁCTICA N° 03**

**NOMBRE COMPLETO:** EMILIANO GARCIA OLVERA

**N° de Cuenta:** 314256009

**GRUPO DE LABORATORIO:** 13

**GRUPO DE TEORÍA:** 06

**SEMESTRE 2026-2**

**FECHA DE ENTREGA LÍMITE:** 05/FEBRERO/2026

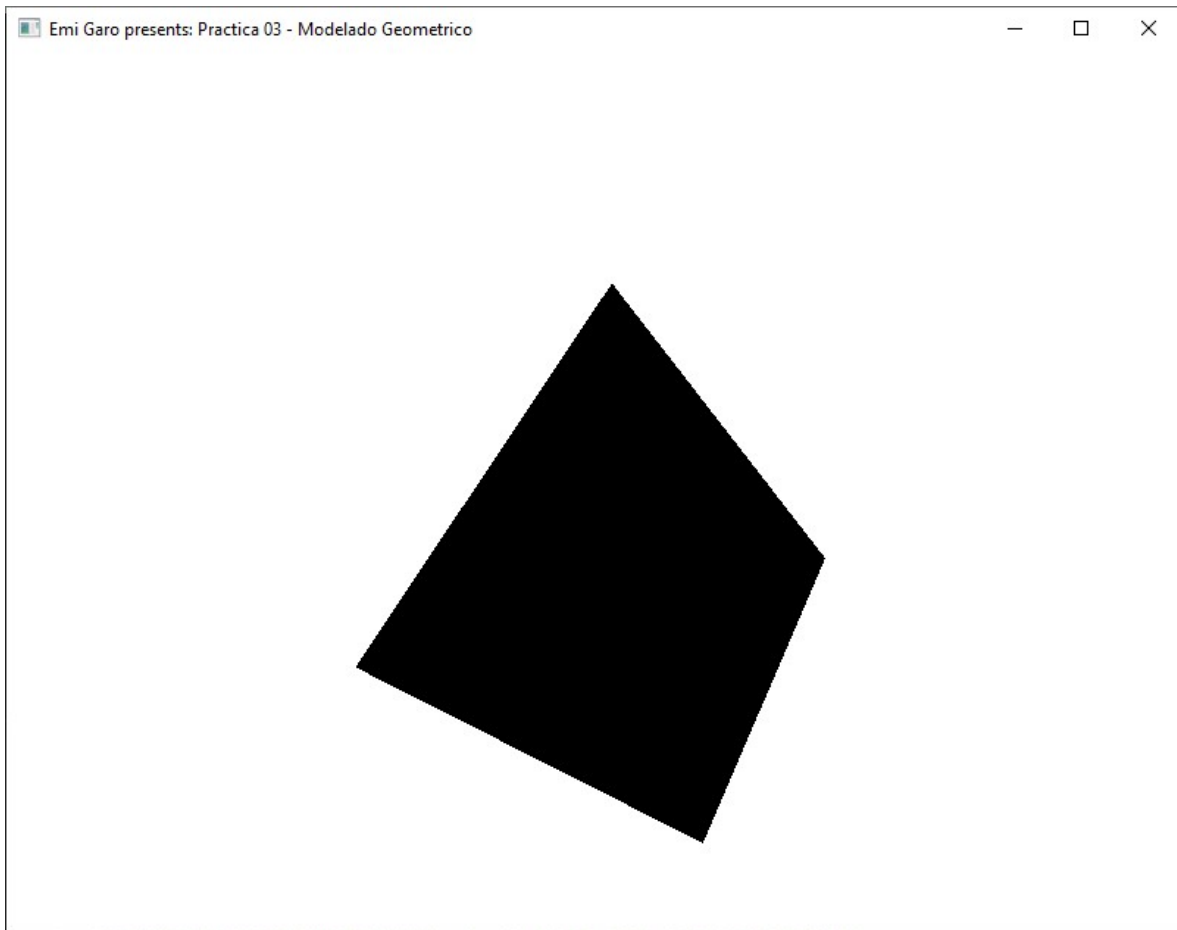
**CALIFICACIÓN:** \_\_\_\_\_

## Generar una pirámide rubik (pyraminx) de 9 pirámides por cara.

Primero generamos una piramide negra que usaremos de base.

```
// Base Pyramid
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
                  glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
                  glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();// Piramide Cuadrangular
```

Notese que cambiamos las coordenadas (0,0,0), esto nos hara la vida mas facil despues.



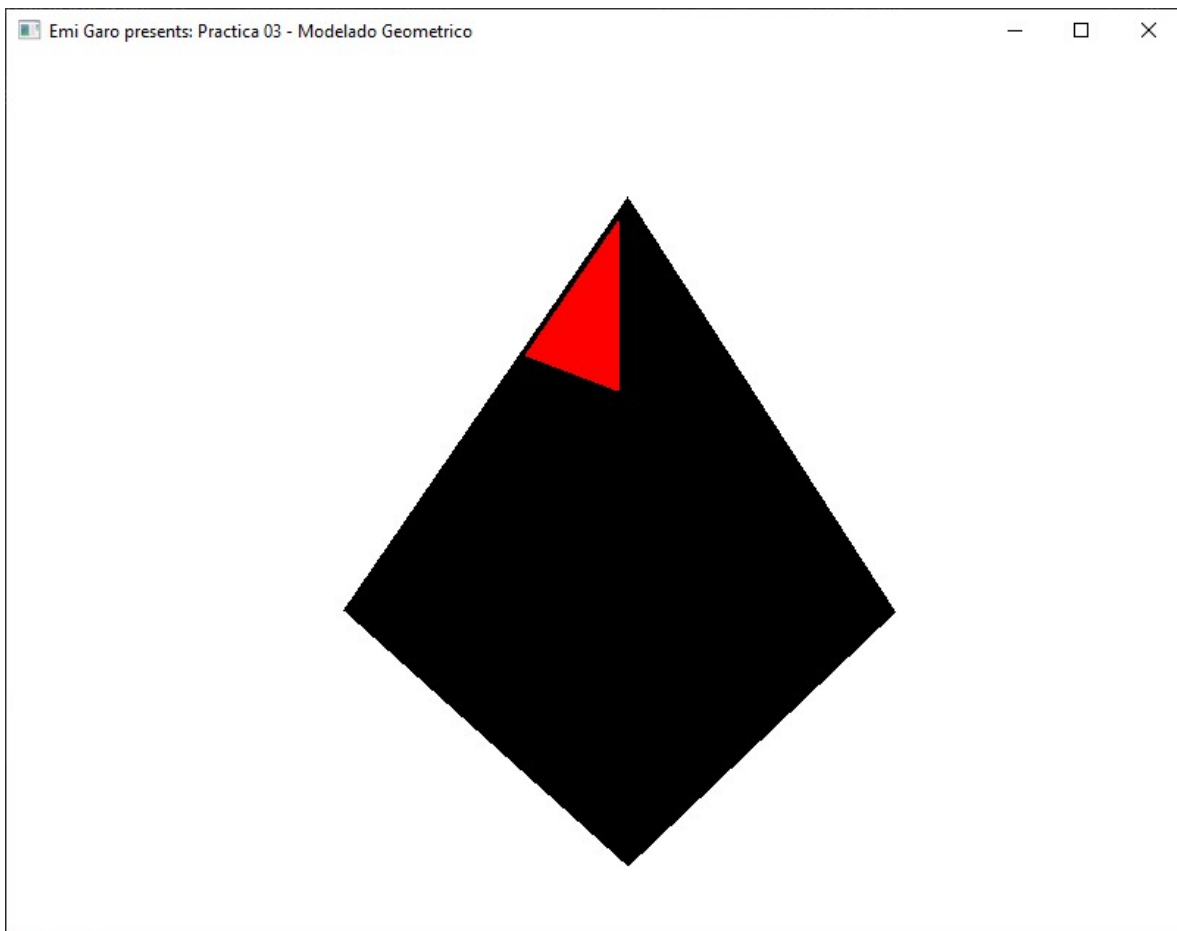
No parece muy tridimensional nuestra piramide todavia pero en unos momentos cambiara.

A continuacion viene la parte mas dificil de la practica (dificil como en tedioso). Incrustar cada pequeña mini piramide de la primera cara.

Despues de prueba y error las siguientes variables parecen dar un resultado satisfactorio para la primera piramide roja.

```
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(0.0f, 0.65f, 0.05f));  
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,  
                    glm::value_ptr(model));  
glUniformMatrix4fv(uniformView, 1, GL_FALSE,  
                    glm::value_ptr(camera.calculateViewMatrix()));  
color = glm::vec3(1.0f, 0.0f, 0.0f);  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
meshList[4]->RenderMesh();
```

Notese tambien que escalamos a 0.54 por lo que piramide pequena es aproximadamente poco mas de un cuarto de la piramide base.



Ya se empieza a ver mejor.

Ahora seguimos con el siguiente lote de piramides.

```
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(-0.3f, 0.0f, 0.37f));  
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
```

```

        glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
        glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();

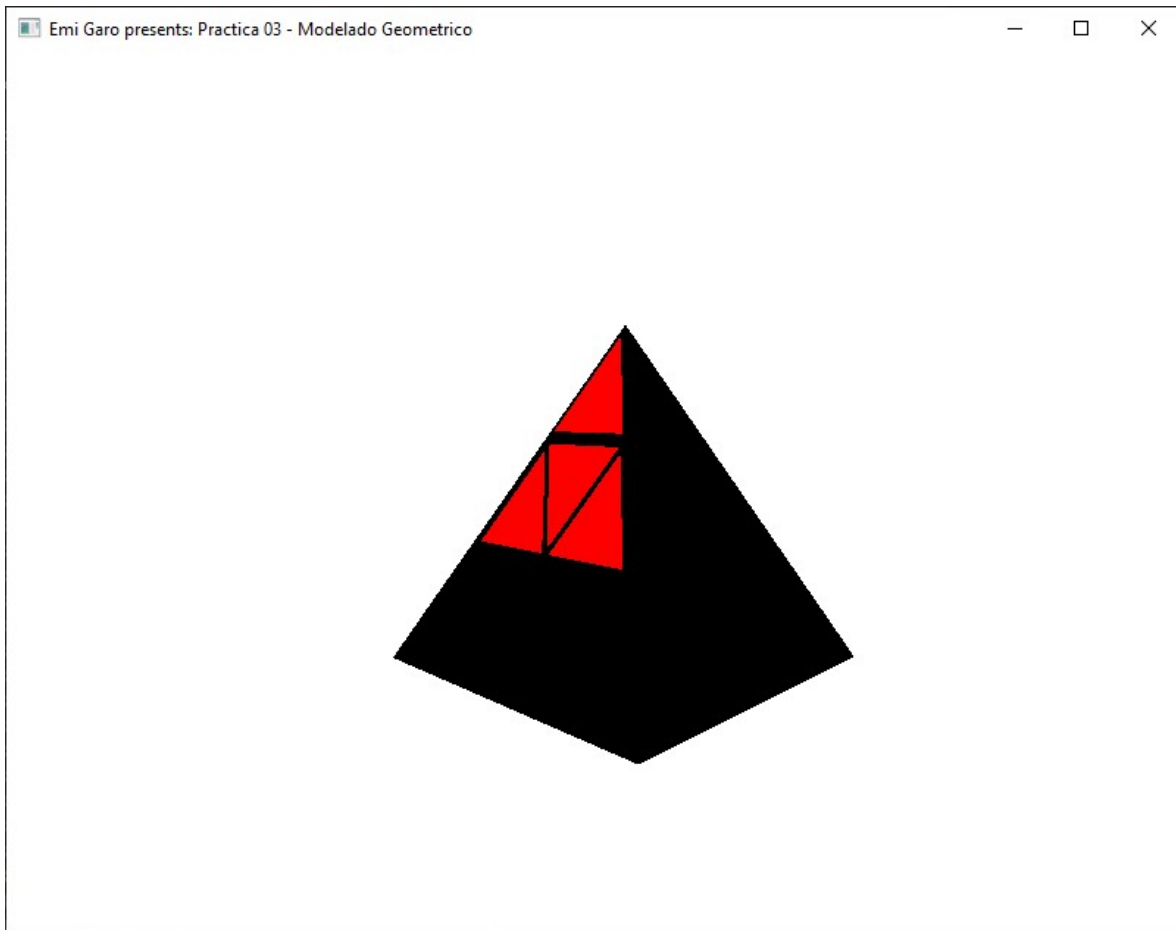
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -0.07f, 0.41));
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
model = glm::rotate(model, 127 * toRadians,
        glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
        glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
        glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.3f, 0.0f, 0.37));
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
        glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
        glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();

```

Parece un poco mas complicado pero basicamente cada vez que vemos `model = glm::mat4(1, 0)`, estamos iniciando el proceso de creacion de una nueva piramide y cada vez que vemos `meshList[4]->RenderMesh()` terminamos con dicha piramide, y comenzamos otra.

Otro dato a resaltar en el codigo es que ahora tenemos una piramide con la siguiente sentencia `model = glm::rotate(model, 127 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f))`. Esta se ocupara de rotar aquellas piramides que necesitan estar boca arriba. Por lo que tendriamos un resultado como el siguiente.



Se va viendo mejor, no?

Ahora para terminar esta cara simplemente repetimos el proceso anterior una vez mas.

```

        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(-0.65f, -0.65f, 0.7));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(1.0f, 0.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(-0.32f, -0.73f, 0.76));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        model = glm::rotate(model, 127 * toRadians, glm::vec3(1.0f, 0.0f,
0.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(1.0f, 0.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));

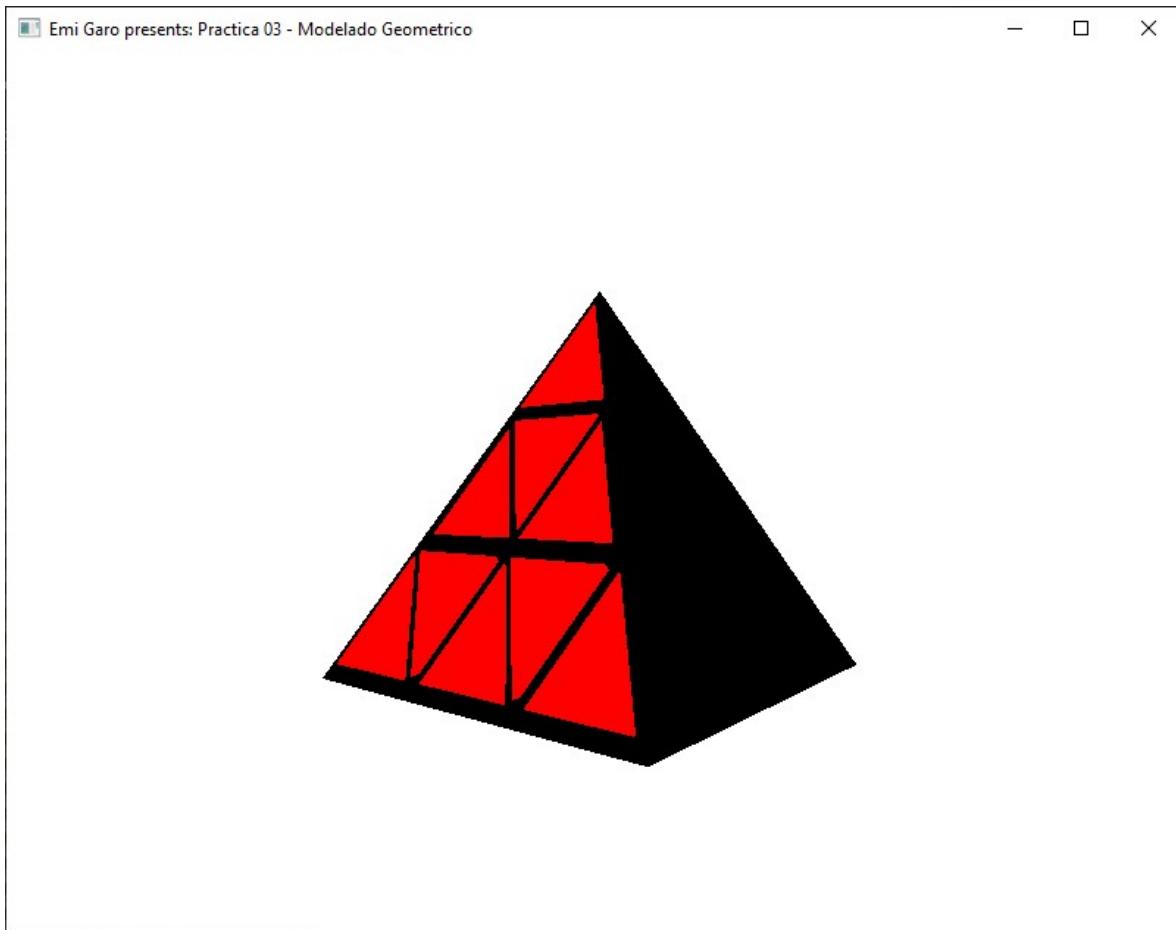
```

```

        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.0f, -0.65f, 0.7));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(1.0f, 0.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.32f, -0.73f, 0.76));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        model = glm::rotate(model, 127 * toRadians, glm::vec3(1.0f, 0.0f,
0.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(1.0f, 0.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.65f, -0.65f, 0.7));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(1.0f, 0.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();

```

Y deberia de quedarnos algo asi.



Ya tenemos una cara completada.

Y las buenas noticias son que ahora podemos ahorrarnos algo de trabajo para completar las demas. Por ejemplo la cara Izquierda puede ser completada simplemente intercambiando coordenadas. Digamos que la primera piramide roja que dibujamos estaba en (0,1, 1), ahora simplemente podemos intercambiar la componente z por x. De esta manera todas las caras que estan en frente pasaran a el lado izquierdo. El codigo luce para esta cara luce asi:

```
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.05f, 0.65f, 0.0));
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();
//2
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.41f, -0.07f, 0.));
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
```

```

        model = glm::rotate(model, -127 * toRadians, glm::vec3(0.0f, 0.0f,
1.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.37f, 0.0f, -0.3));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.37f, 0.0f, 0.3));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        // 3
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.7, -0.65f, -0.65));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.7f, -0.65f, 0.0));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.7, -0.65f, 0.65));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);

```

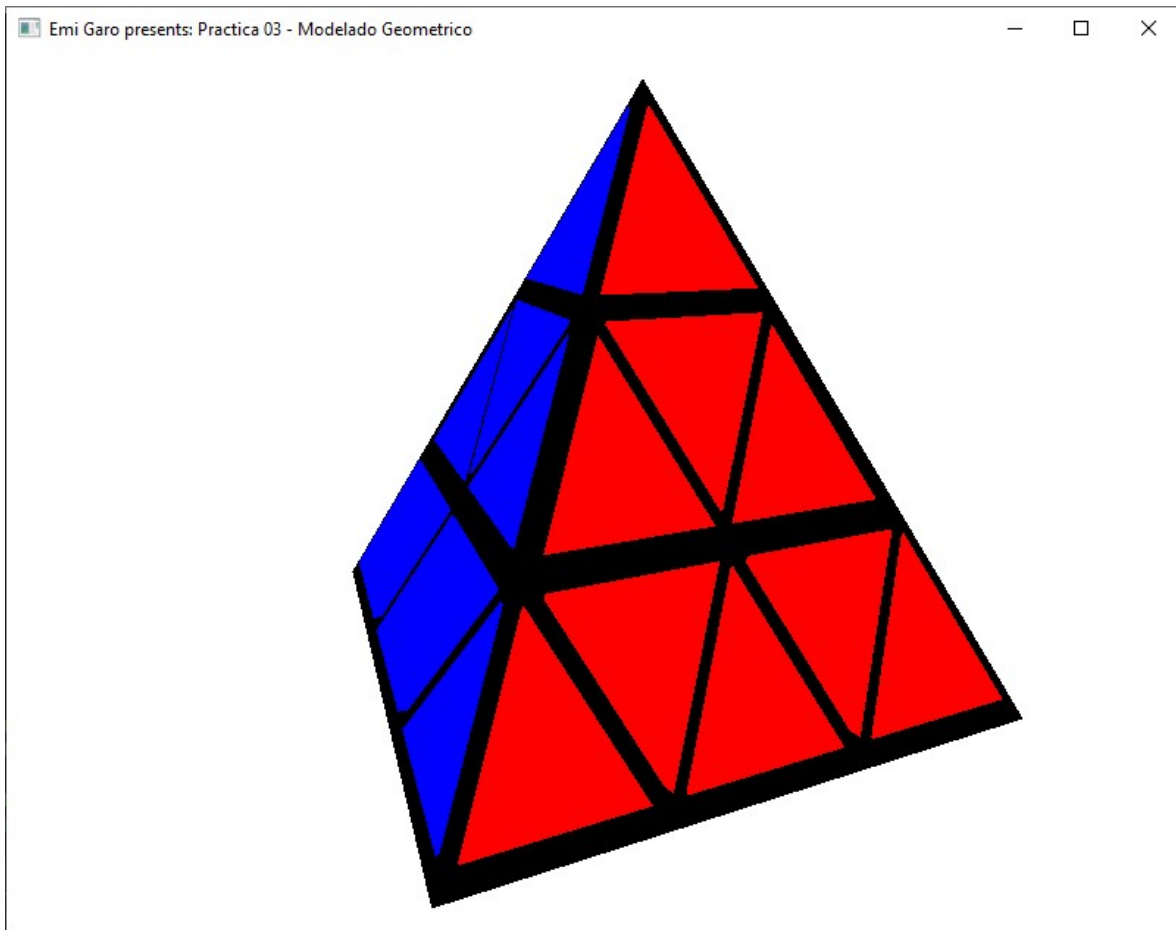


```

        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.76, -0.73f, -0.32));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        model = glm::rotate(model, -127 * toRadians, glm::vec3(0.0f, 0.0f,
1.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.76, -0.73f, 0.32));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        model = glm::rotate(model, -127 * toRadians, glm::vec3(0.0f, 0.0f,
1.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();

```

Y el resultado así:



Ahora si pusiste atencion, te daras cuenta que puedes hacer exactamente lo mismo para la cara derecha. Y si eres mas astuto te daras cuenta que es incluso mas facil. Simplemente invertimos el valor de nuestra componente en x de positivo a negativo y el codigo de esta cara quedaria asi:

```
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.05f, 0.65f, 0.0));
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();
//2
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.41f, -0.07f, 0.0));
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
model = glm::rotate(model, -127 * toRadians, glm::vec3(0.0f, 0.0f,
1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
```

```

        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.37f, 0.0f, -0.3));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.37f, 0.0f, 0.3));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        // 3
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.7, -0.65f, -0.65));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.7f, -0.65f, 0.0));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.7, -0.65f, 0.65));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.76, -0.73f, -0.32));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));

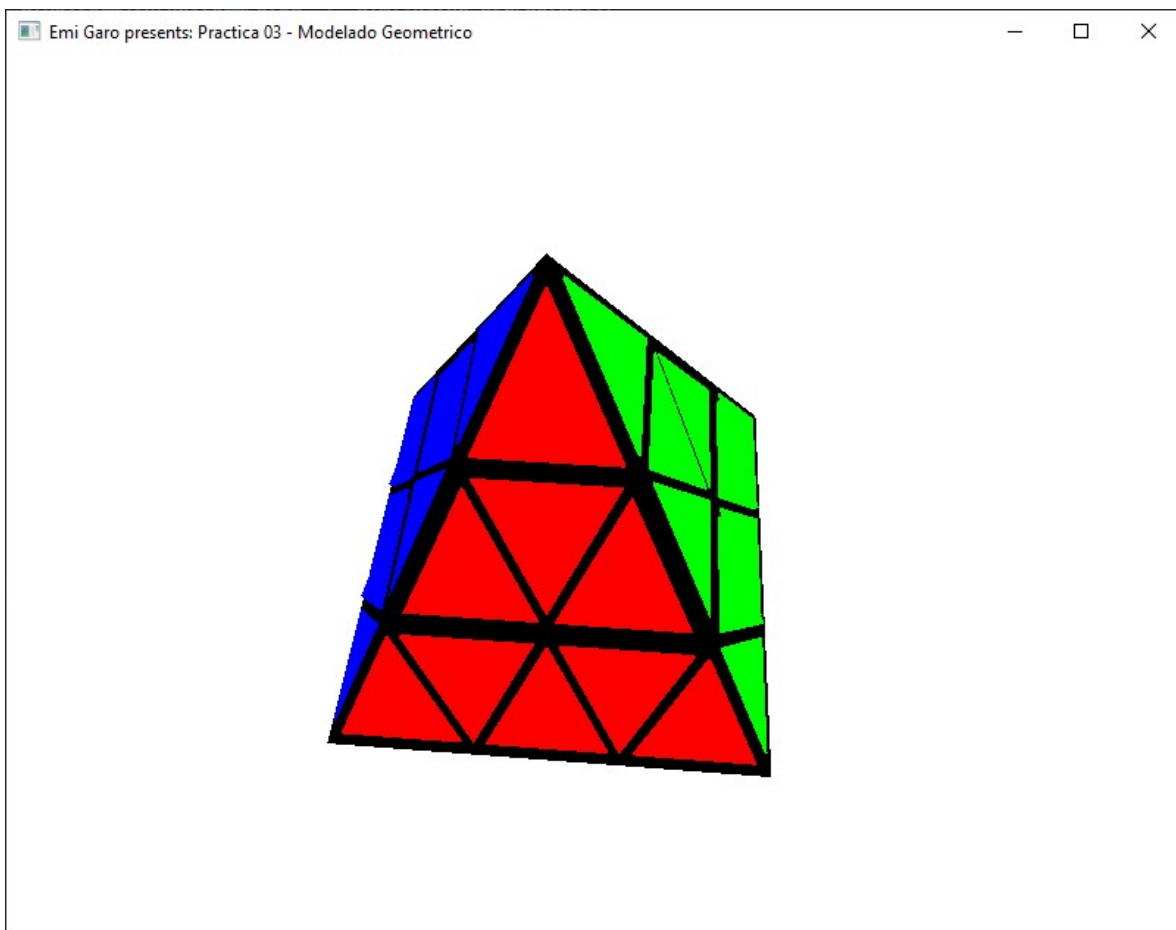
```

```

        model = glm::rotate(model, -127 * toRadians, glm::vec3(0.0f, 0.0f,
1.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.76, -0.73f, 0.32));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        model = glm::rotate(model, -127 * toRadians, glm::vec3(0.0f, 0.0f,
1.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(0.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();

```

Notemos tambien que esta vez la rotacion es negativa y cambiamos el axis de rotacion, de lo contrario no obtendriamos la rotacion esperada, y por ende el resultado esperado, o sea el siguiente:



Finalmente y de manera similar para obtener la ultima cara invertimos los valores de la primera cara que calculamos, casi como si empujaramos cada piramide siguiendo la tercera ley de newton. Para terminar con un codigo como el siguiente:

```
// 1
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.65f, -0.05f));
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();
//2
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-0.3f, 0.0f, -0.37f));
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -0.07f, -0.41f));
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
model = glm::rotate(model, -127 * toRadians, glm::vec3(1.0f, 0.0f,
0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.3f, 0.0f, -0.37f));
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();
// 3
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-0.65f, -0.65f, -0.7f));
model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
```

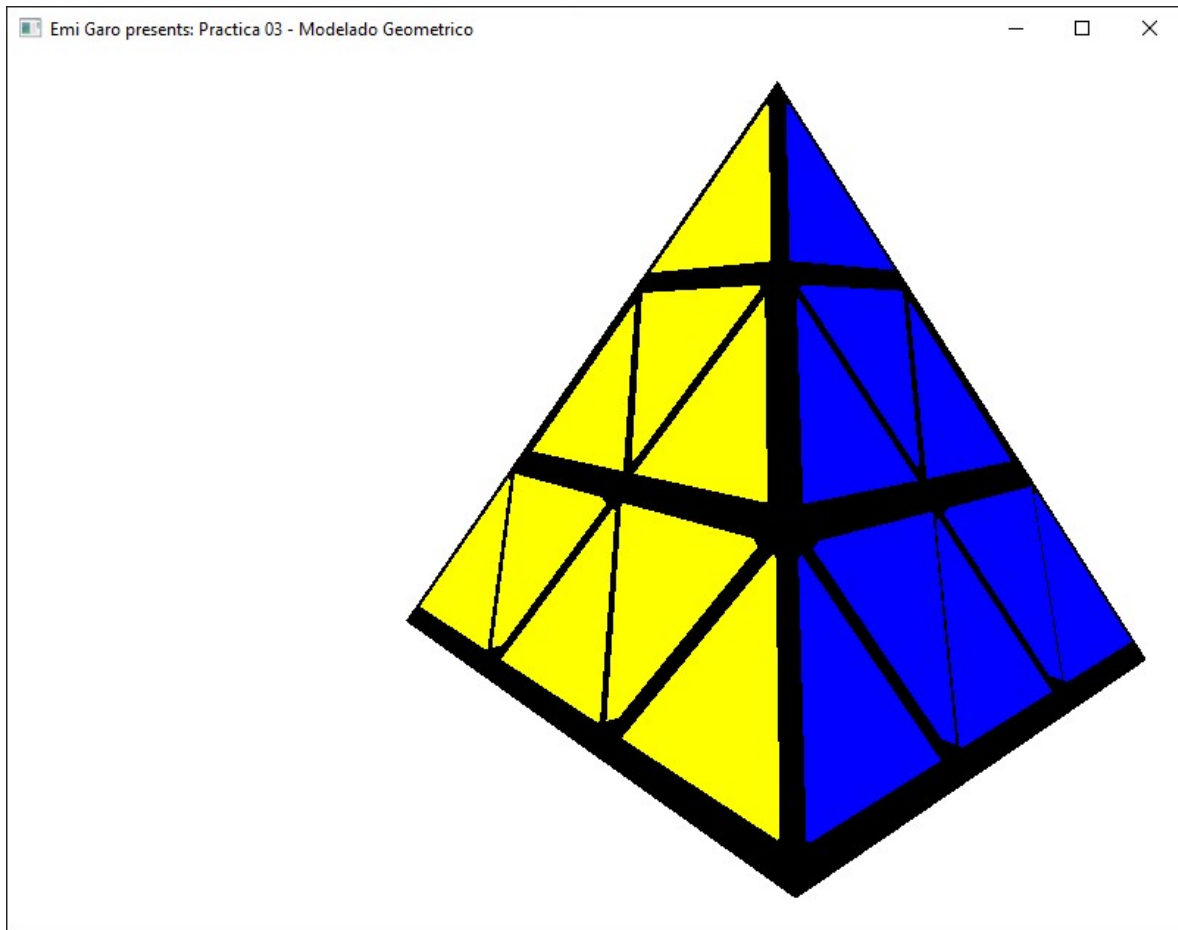
```

        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(-0.32f, -0.73f, -0.76));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        model = glm::rotate(model, -127 * toRadians, glm::vec3(1.0f, 0.0f,
0.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(1.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.0f, -0.65f, -0.7));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(1.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.32f, -0.73f, -0.76));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        model = glm::rotate(model, -127 * toRadians, glm::vec3(1.0f, 0.0f,
0.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(1.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();
        model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.65f, -0.65f, -0.7));
        model = glm::scale(model, glm::vec3(0.54f, 0.54f, 0.54f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
        color = glm::vec3(1.0f, 1.0f, 0.0f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[4]->RenderMesh();

```

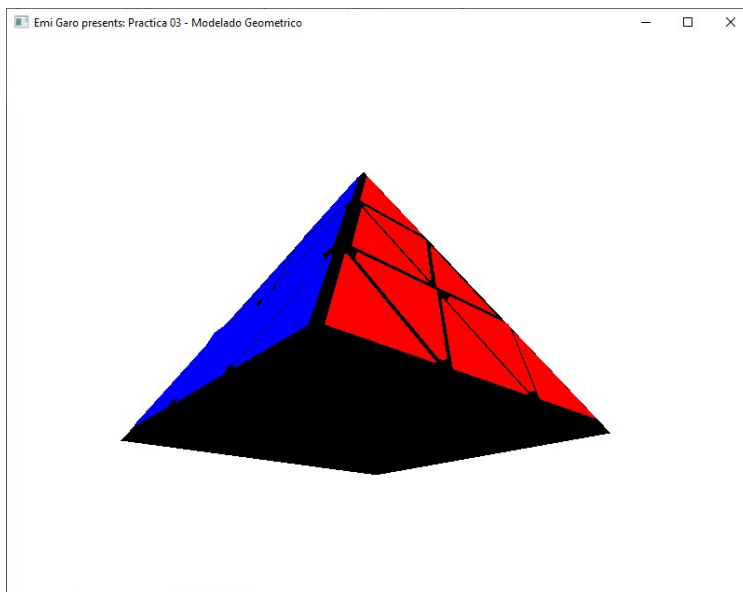
Notemos que volvemos a rotar una vez mas en la direccion contraria para que las piramides queden paralelas a la piramide base.

Lo que finalmente nos da el resultado que queriamos.



Y hemos terminado!

Bonus:



Nuestra piramide vista desde abajo.

## **Conclusión**

A pesar de parecer un ejercicio extremadamente difícil a primera vista. Hemos visto que entender las transformaciones en el espacio tridimensional realmente nos ha ayudado a ser rápidos, eficientes y ordenados. Además de permitirnos reutilizar trabajo anterior para poder realizar transformaciones simples para completar las demás caras de la pirámide.

Por lo que si entendimos lo que realizamos en esta práctica. Ya tenemos nociones básicas para realizar simples transformaciones (trasladar, escalar y rotar) en primitivas modeladas a través de código embebido en nuestro programa y luego siendo renderizadas utilizando la API de OpenGL.

Finalmente ahondamos más en nuestros conocimientos acerca del pipeline de OpenGL y como pasamos desde simples datos como vértices y caras (formadas a partir de índices a dichos vértices) hasta ser modificados por medio de transformaciones de matrices de dichas primitivas, fuimos introducidos a las transformaciones que estos pasan para poder simular una cámara que controlamos y para finalmente ser renderizados en una superficie 2D como lo es nuestra pantalla.

## **Referencias:**

- Apuntes de la clase de teoría y laboratorio de “Computación Gráfica e Interacción Humano Computadora”